property. That's how `dispatch` works.

However, the other properties of each action are different. The shape of each action's data depends on exactly what kind of action the object is. A hold ticket action needs the seat and row number of a ticket, but the set tickets-to-buy count action only needs the new number of tickets to buy. TypeScript needs to know what data is available when we execute each action so it can say whether the code is type-safe.

We could handle this by creating a parent class that only has a `type` property and then having subclasses, but then we wouldn't be able to use object literal syntax to create objects, and I'd like to have the convenience of `{ type: "AddFilter" }`.

Happily, TypeScript includes a feature called a *string literal type* that allows us to declare interfaces based on literal string values.

For each individual action, we declare a TypeScript `interface` where the `type` property is set to a literal string value and the other properties are added as needed. Then we declare an `VenueAction` type to be a combination of all the types we've declared. `VenueAction` is a *union type*, which means that an object can be a `VenueAction` if it matches any of the types that make up the union. Now when we use these objects, TypeScript will pattern match based on the value of the `type` property to determine which interface the object matches.

To use these types, we declare the `action` argument of the `reducer` to be of type `VenueAction`, meaning it can be one of any of the component types of the `VenueAction` union type.

If we just keep the object as type `VenueAction`, we can only access properties that are common across all of the component types, which in this case, means only `type`.

That's not very useful, but TypeScript is really smart about this. In our `reducer` function, when we switch based on `action.type`, TypeScript infers the interface being used. In other words, when we branch into, say, the `setTicketsToBuy` branch,